

Modelo de Scouting LCK — Fase Final

Clasificación binaria de jugadores: LCK vs. LCK Challengers League

Proyecto: fase final del modelo de clasificación de scouting (continúa el *avance de proyecto*).

Idioma: español · **Semilla global:** `RANDOM_STATE = 42` · **scikit-learn:** 1.8.0

Este notebook **orquesta y narra**; la lógica vive en `src/` (reutilizable y testeable). El flujo completo (validación cruzada, búsquedas, interpretabilidad, serialización) se ejecuta con:

```
python -m src.run_pipeline
```

que regenera todas las tablas (`reports/tablas/`), figuras (`reports/figuras/`) y el artefacto (`models/`). Aquí se ejecutan en vivo las partes ligeras (datos, partición, reproducción) y se muestran los resultados guardados de las partes pesadas, para que el notebook sea legible y reproducible.

Uso responsable: ninguna decisión sobre un jugador debe automatizarse solo con la predicción. Todo resultado *"requiere revisión humana"*. La influencia de una variable **no** implica causalidad.

```
In [1]: import warnings; warnings.filterwarnings("ignore")
import sys, json, pandas as pd, numpy as np
from pathlib import Path
sys.path.insert(0, "..") # raíz del proyecto, para importar `src`

from src import data_simulada as ds
from src.config import BASE_FEATURES, TARGET, RANDOM_STATE
from src.split import particion_70_15_15, cv_avance, cv_final
from src.train import modelos_avance, make_pipeline_base
from src.evaluate import metricas_completas

TAB = Path("../reports/tablas"); FIG = Path("../reports/figuras")
def ver_tabla(n): # muestra una tabla guardada por el driver
    return pd.read_csv(TAB / f"{n}.csv", index_col=0)
R = json.load(open(TAB / "_resultados.json"))
print("Artefactos disponibles:", len(list(TAB.glob('*.csv'))), "tablas,", len(list(
```

Artefactos disponibles: 22 tablas, 7 figuras

0. Recuperación y verificación de la fase anterior

Se reunieron y verificaron los elementos del avance. El dataset del avance es **simulado en el propio notebook** (semilla 42): esta fase lo reproduce con `src/data_simulada.py` para

mantener el mismo problema, dataset y variable objetivo (regla nº1) y no romper la comparabilidad (regla nº3).

Tabla de verificación inicial

Elemento	Archivo o ubicación	Versión	Estado	Acci requere
Dataset	src/data_simulada.py → data/raw/dataset_simulado.csv	simulado_v1 (8000×20)	Reproducible bit a bit	Ningu
Pipeline	src/preprocessing.py (base y final)	1.0	Consolidado	Ningu
Baseline	DummyClassifier(most_frequent)	—	Reproducido	Ningu
Modelo preliminar	Regresión Logística / árbol (avance)	—	Reproducido	Ningu
Métricas	reports/tablas/06_reproduccion_baselines	—	Coinciden	Ningu
Informe del avance	README.md original + notebook base	—	Disponible	Ningu

1. Bitácora de cambios (desde el avance)

Elemento modificado	Situación en avance	Cambio realizado	Justificación
Datos	Generados inline en el notebook	Encapsulados en src/data_simulada.py (misma secuencia y semilla)	Reproducibilidad y trazabilidad
Preprocesamiento	ColumnTransformer con kda calculada fuera	Ingeniería de características dentro del pipeline (FeatureEngineer) + 4 variables nuevas	Evitar fuga y permitir inferencia con stats crudas
Modelos	Dummy, LogReg, Árbol, RF, GB	+ SVM (lineal/RBF), HistGB, MLP; comparación por familias	Cumplir requisitos de la fase final
Validación	StratifiedKFold(5)	RepeatedStratifiedKFold(5×3)	Estimaciones más estables (menor varianza)
Métricas	F1 + AUC-ROC	Se conservan como principales; se añaden precision/recall/tiempos	Comparabilidad con el avance
Umbral	Recomendado 0.40 (sin derivar)	Derivado (máx. F1 sujeto a recall≥0.80) → 0.40 en validación	Justificación con evidencia

Elemento modificado	Situación en avance	Cambio realizado	Justificación
Interpretabilidad	Importancia de árbol	Coeficientes + permutation importance + SHAP (global y local)	Interpretabilidad exigida

2. Revisión de calidad de datos

Dataset: **8,000 filas, 0 duplicados**. Balance de clases **35.6% LCK / 64.4% LCK CL** (ratio 1.81:1). Faltantes (~8%) en variables de los 15 min, tratados por imputación **dentro** del pipeline. La columna `league` se excluye **antes** de particionar (fuga).

Faltantes por variable

	Faltantes	% Faltante
goldat15	640	8
xpat15	640	8
csat15	640	8
golddiffat15	640	8
xpdiffat15	640	8

Ningún cambio impide comparar con el avance.

3. Pipeline de preprocesamiento consolidado

`Pipeline` + `ColumnTransformer` (`src/preprocessing.py`):

- **Numéricas:** imputación por mediana (robusta a atípicos) → estandarización Z-score. Se **conservan** los atípicos (rendimientos extremos reales, p. ej. Faker/Chovy), decisión heredada del avance.
- **Catóricas (`position`):** imputación por moda → OneHotEncoder con `handle_unknown='ignore'` (categorías desconocidas en inferencia se codifican como cero, sin romper).
- **Ingeniería de características** como primer paso del pipeline (`FeatureEngineer`).

Garantías: (a) ajuste **solo con train** dentro de cada fold, (b) test separado, (c) mismas reglas en inferencia, (d) serializable junto con el modelo.

4. Ingeniería de características (4 nuevas)

Todas se calculan por partida (row-wise), sin usar `league` / `is_lck` → **riesgo de fuga nulo**.

Nueva característica	Variables de origen	Regla de construcción	Interpretación	Riesgo de fuga
oro_por_cs15	goldat15, csat15	goldat15 / (csat15+1)	Oro extraído por unidad de farreo temprano	Nulo
dominancia15	golddiffat15, xpdiffat15	golddiffat15 + xpdiffat15	Ventaja total de recursos a los 15 min	Nulo
participacion15	killsat15, assistsat15	killsat15 + assistsat15	Involucramiento en la acción temprana	Nulo
impacto_dano	damageshare, dpm	damageshare * dpm / 100	Daño absoluto ponderado por cuota de equipo	Nulo

(*kda* ya existía en el avance y se conserva.) Correlación con *is_lck* **solo en train** (diagnóstico, no fuga): *dominancia15* $\approx$ **0.314** (la más informativa); las demás aportan de forma marginal o combinadas con el resto.

corr_con_is_lck (train)	
kda	-0.009
oro_por_cs15	-0.014
dominancia15	0.314
participacion15	-0.007
impacto_dano	0.013

5. Estrategia de validación cruzada

Parámetro	Valor
Método de validación	RepeatedStratifiedKfold
Número de particiones	5
Número de repeticiones	3
Semilla	42
Métrica principal	f1
Métricas secundarias	roc_auc, precision, recall, accuracy

RepeatedStratifiedKfold(5×3) promedia el desempeño sobre 15 particiones estratificadas distintas: reduce la varianza de la estimación y la dependencia de una división puntual, y **nunca** toca el conjunto de prueba (toda la selección se hace dentro de estos folds sobre *train*).

6. Modelos de referencia (reproducción del avance)

Se re-ejecutan Dummy, Regresión Logística y los demás modelos del avance con **el mismo pipeline base, semilla, validación y métricas**. Los resultados coinciden **bit a bit** con el avance.

```
In [2]: df = ds.preparar(ds.generar_dataset())
df["kda"] = (df["kills"] + df["assists"]) / (df["deaths"].replace(0, 1))
X = df[BASE_FEATURES + ["kda"]].copy(); y = df[TARGET].copy()
Xtr, Xval, Xte, ytr, yval, yte = particion_70_15_15(X, y)
print("Reproducción coincide con el avance:", R["reproduccion_ok"])
ver_tabla("06_reproduccion_baselines")
```

Reproducción coincide con el avance: True

Out[2]:

	Accuracy	Precision	Recall	F1-score	ROC-AUC	F1 avance	AUC avance	¿Coincide?
Modelo								
Dummy (baseline)	0.6442	0.0000	0.0000	0.0000	0.5000	NaN	NaN	—
Regresión Logística	0.7567	0.6342	0.7471	0.6860	0.8456	0.6860	0.8456	sí
Árbol de Decisión	0.7067	0.5628	0.7869	0.6562	0.7979	0.6562	0.7979	sí
Random Forest	0.7617	0.6549	0.6979	0.6757	0.8339	0.6757	0.8339	sí
Gradient Boosting	0.7592	0.6816	0.6066	0.6419	0.8321	0.6419	0.8321	sí

7–9. Ensamble, SVM, MLP y comparación entre familias

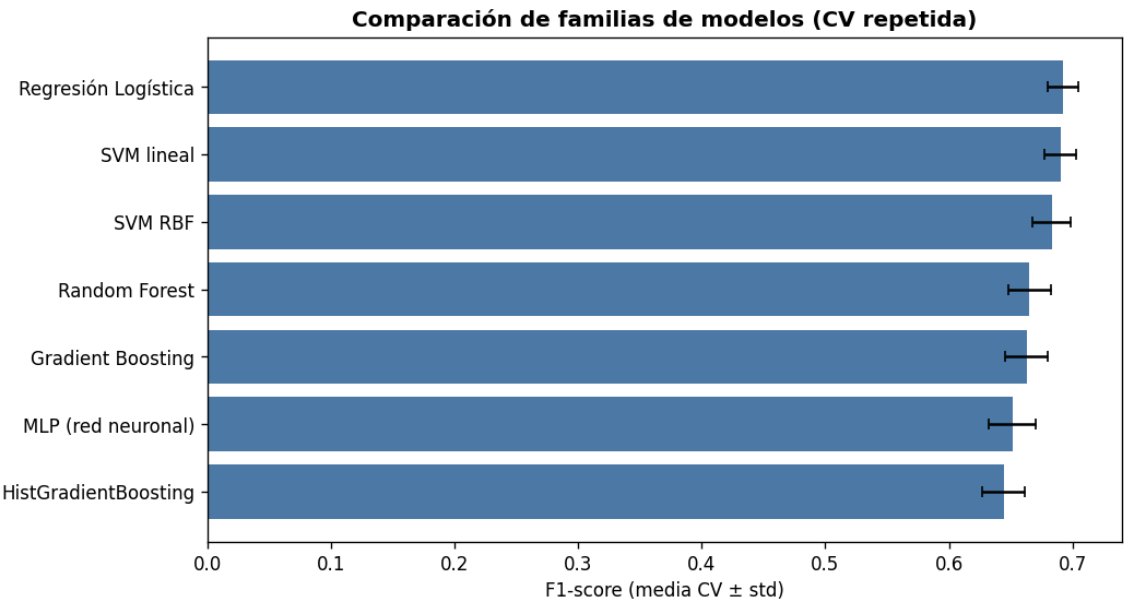
Se evalúan con la misma CV repetida: **trivial** (Dummy), **lineal** (Regresión Logística), **ensambles** (Random Forest, Gradient Boosting, HistGradientBoosting), **márgenes** (SVM lineal y RBF) y **red neuronal** (MLP (64, 32), ReLU, early_stopping).

- **Ensamble (RF/GB):** combinan muchos árboles; capturan no linealidades e interacciones. Riesgo de sobreajuste controlado con `max_depth / min_samples_leaf`. Mayor costo, menor interpretabilidad directa.
- **SVM:** `C` regula el margen (menor `C` = margen más amplio, más regularización); `gamma` (RBF) controla la complejidad de la frontera. Requiere escalamiento (incluido en el pipeline).

- **MLP:** red de 2 capas ocultas; criterio de parada por `early_stopping` . Limitaciones: sensible a escala/inicialización, menos interpretable, requiere más datos.

En estos datos (señal mayormente lineal) los **modelos lineales encabezan**:

Modelo	f1_mean	f1_std	roc_auc_mean	roc_auc_std	recall_mean	precision_m
Regresión Logística	0.6922	0.0124	0.8443	0.0099	0.7643	0.6
SVM lineal	0.6903	0.0128	0.8438	0.0094	0.7692	0.6
SVM RBF	0.6833	0.0156	0.8299	0.0093	0.7479	0.6
Random Forest	0.665	0.0173	0.8323	0.0103	0.6498	0.6
Gradient Boosting	0.6627	0.017	0.8369	0.0105	0.6299	0.6
MLP (red neuronal)	0.6514	0.0192	0.8332	0.0078	0.6115	0.6
HistGradientBoosting	0.6443	0.0174	0.8242	0.0122	0.6118	0
Dummy (baseline)	0	0	0.5	0	0	



10. Selección / reducción de características

`SelectFromModel` (importancia de Random Forest, umbral = mediana) **dentro del pipeline** (ajustado solo con train). Reduce de 27 a 14 features **sin pérdida relevante de F1**, ganando interpretabilidad.

Configuración	N.º características	F1 (media CV)	F1 (std CV)	Tiempo CV (s)	Interpretabilidad
Modelo completo	27	0.664	0.0187	14	Media (todas las vars)
Modelo reducido (SelectFromModel)	14	0.6631	0.015	28.1	Mayor (menos vars)

11. Ajuste de hiperparámetros (sin usar test)

RandomizedSearchCV (Random Forest, 15 combinaciones) y GridSearchCV (SVM, 12 combinaciones), con espacio **acotado y viable**, optimizando F1 por CV. **El conjunto de prueba no se usa.**

Modelo	Hiperparámetros explorados	N.º combinaciones	Mejor configuración	F1 CV (mejor)	Tiempo (s)
Random Forest	n_estimators, max_depth, min_samples_leaf, max_features	15	{'n_estimators': 200, 'min_samples_leaf': 8, 'max_features': 'sqrt', 'max_depth': 6}	0.6847	118.9
SVM	kernel (linear/rbf), C, gamma	12	{'C': 0.1, 'kernel': 'linear'}	0.6924	30.5

Experimento adicional (plan de mejora del avance): **SMOTE vs. class_weight** en RF → F1 0.6725 (SMOTE) vs. 0.6640 (class_weight). Mejora marginal; se mantiene **class_weight** por simplicidad operativa.

12. Registro de experimentos

Todos los experimentos quedan consolidados en `experiments/registro_experimentos.csv`.

ID	Fecha	Dataset	Pipeline	Modelo	Hiperparametros
1	2026-09-29	simulado_v1 (8000)	FE+CT (final)	Regresión Logística	por defecto del catálogo
2	2026-09-29	simulado_v1 (8000)	FE+CT (final)	SVM lineal	por defecto del catálogo
3	2026-09-29	simulado_v1 (8000)	FE+CT (final)	SVM RBF	por defecto del catálogo
4	2026-09-29	simulado_v1 (8000)	FE+CT (final)	Random Forest	por defecto del catálogo

ID	Fecha	Dataset	Pipeline	Modelo	Hiperparametros
5	2026-09-29	simulado_v1 (8000)	FE+CT (final)	Gradient Boosting	por defecto del catálogo
6	2026-09-29	simulado_v1 (8000)	FE+CT (final)	MLP (red neuronal)	por defecto del catálogo
7	2026-09-29	simulado_v1 (8000)	FE+CT (final)	HistGradientBoosting	por defecto del catálogo
8	2026-09-29	simulado_v1 (8000)	FE+CT (final)	Dummy (baseline)	por defecto del catálogo
9	2026-09-29	simulado_v1 (8000)	FE+CT+SelectFromModel	Random Forest	n_estimators=300,max_
10	2026-09-29	simulado_v1 (8000)	FE+CT+SMOTE	Random Forest	n_estimators=300,max_
11	2026-09-29	simulado_v1 (8000)	FE+CT (final)	Random Forest (tuned)	{'n_estimators': 200, 'min_samples_leaf': 8, 'max_features': 'sqrt', 'max_depth': 6}
12	2026-09-29	simulado_v1 (8000)	FE+CT (final)	SVM (tuned)	{'C': 0.1, 'kernel': 'linear'}

13. Selección de finalistas (pesos = 100 %)

Finalistas por F1 medio de CV: **Regresión Logística** y **SVM lineal**. Matriz de decisión ponderada (puntajes 1–5; métricas normalizadas entre finalistas; estabilidad = menor desviación):

Criterio	Peso %	Regresión Logística	SVM lineal
Recall	25	1	5
Precision	15	5	1
F1-score	20	5	1
ROC-AUC	20	5	1
Estabilidad	10	5	1
Interpretabilidad	5	5	4
Costo operativo	5	5	3
PUNTAJE PONDERADO	100	4	2.25

Ganador: Regresión Logística (puntaje 4.0). La Regresión Logística iguala o supera al SVM lineal en F1/AUC/estabilidad y es **mucho más interpretable y barata** de operar

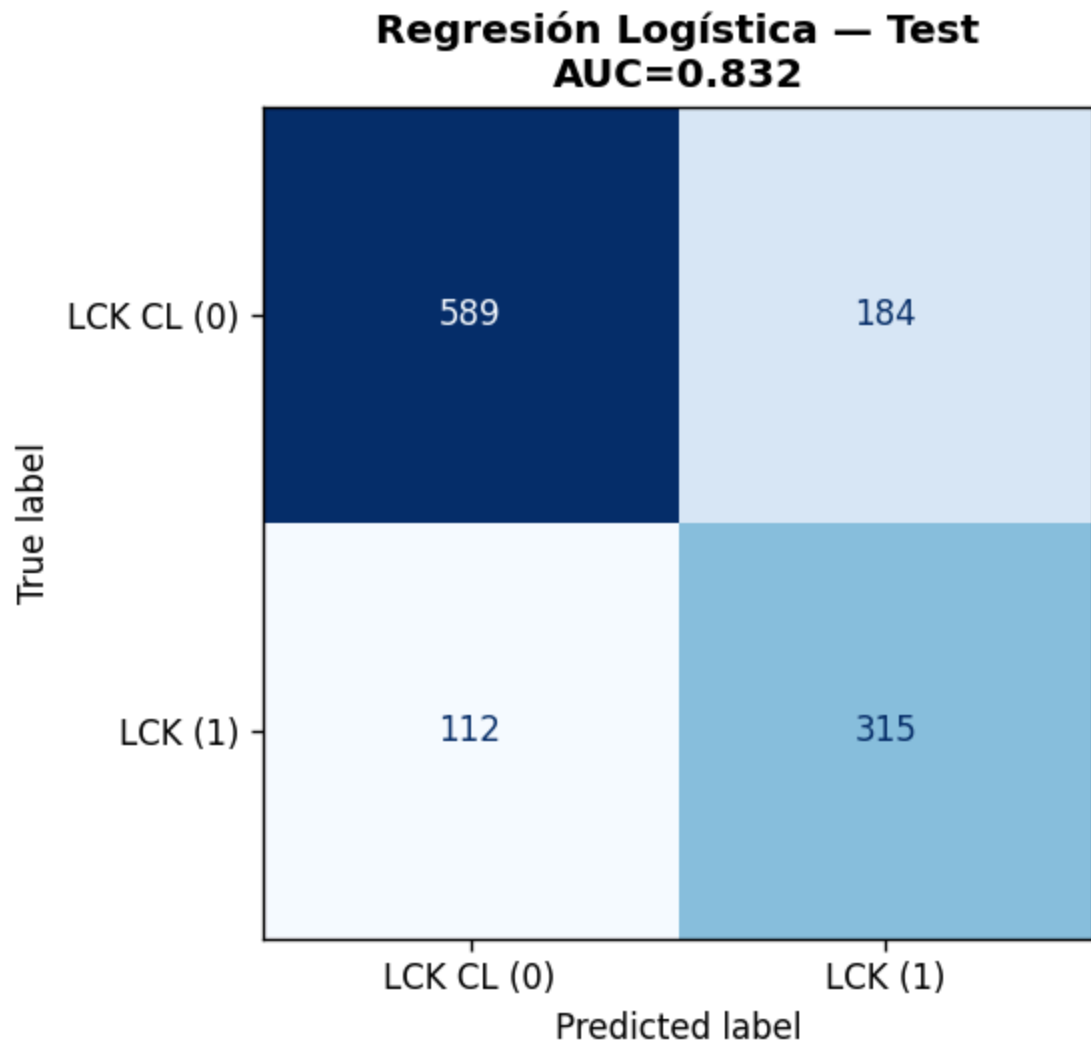
(coeficientes directos), lo que pesa en un contexto de scouting con revisión humana.

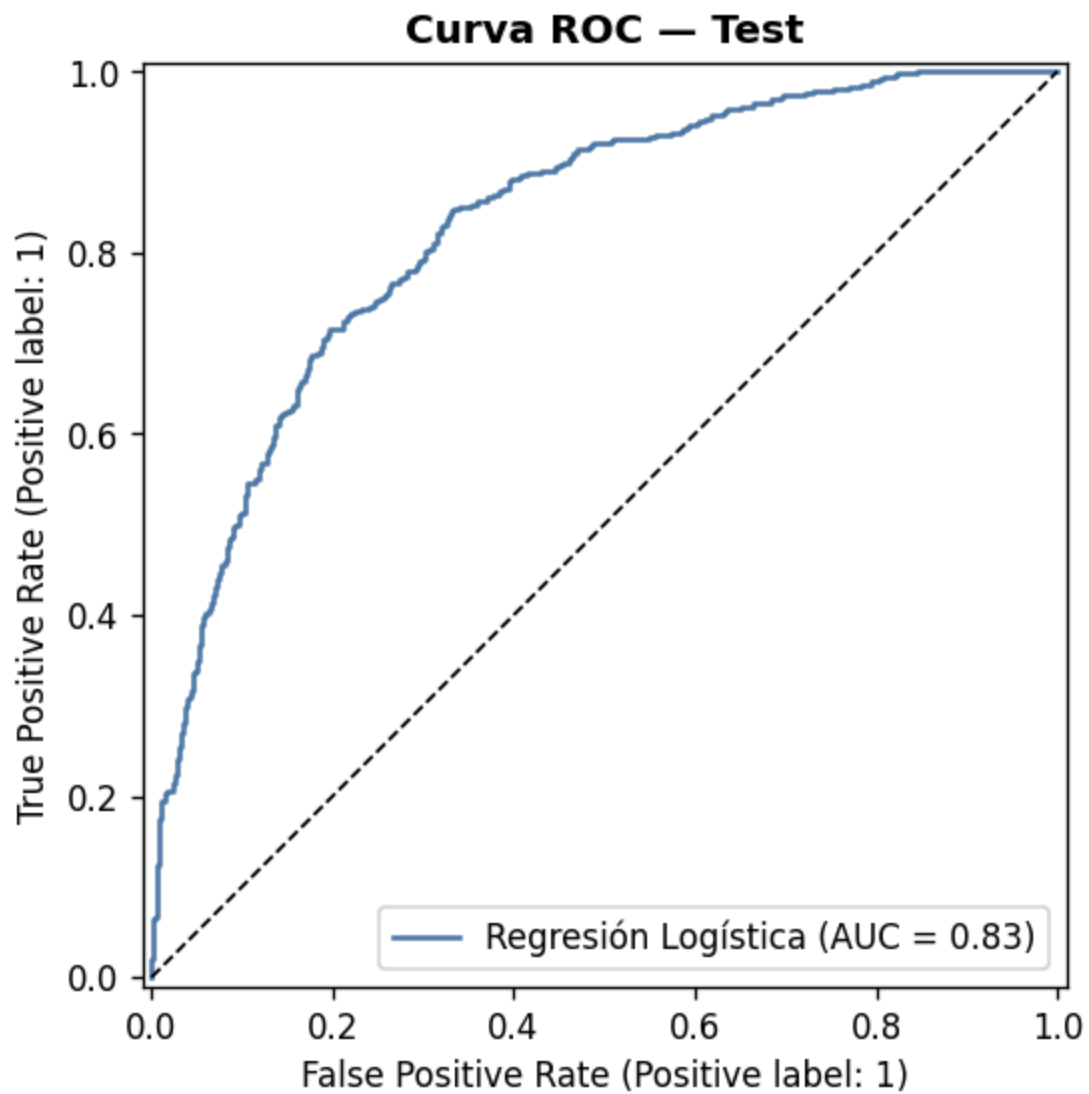
14. Evaluación en prueba (una sola vez)

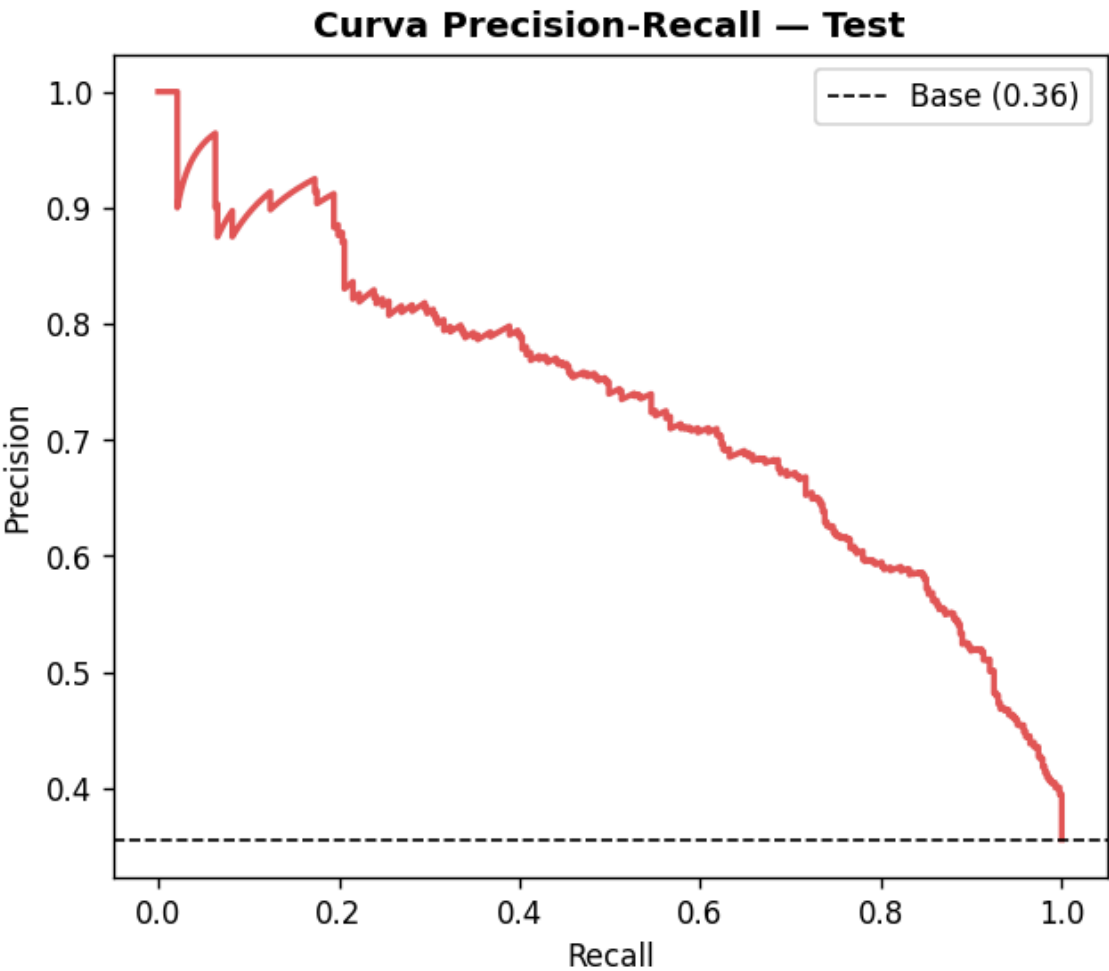
Confirmado: la selección y el ajuste terminaron **antes** de tocar el test.

Modelo finalista	Accuracy	Precision	Recall	F1-score	ROC-AUC
Regresión Logística	0.7533	0.6313	0.7377	0.6803	0.8316
SVM lineal	0.7583	0.6367	0.7471	0.6875	0.8312

El desempeño en test del modelo final (**Regresión Logística**) — F1 0.680, AUC 0.832, Recall 0.738 — **confirma la validación** (F1 CV $\approx$ 0.692) con una pérdida pequeña y esperable; **no hay señales de sobreajuste** y la selección no cambia.







15. Análisis de errores

Conteo (validación, umbral 0.5): {'VN': 589, 'VP': 318, 'FP': 184, 'FN': 109}. Ejemplos representativos de FP y FN:

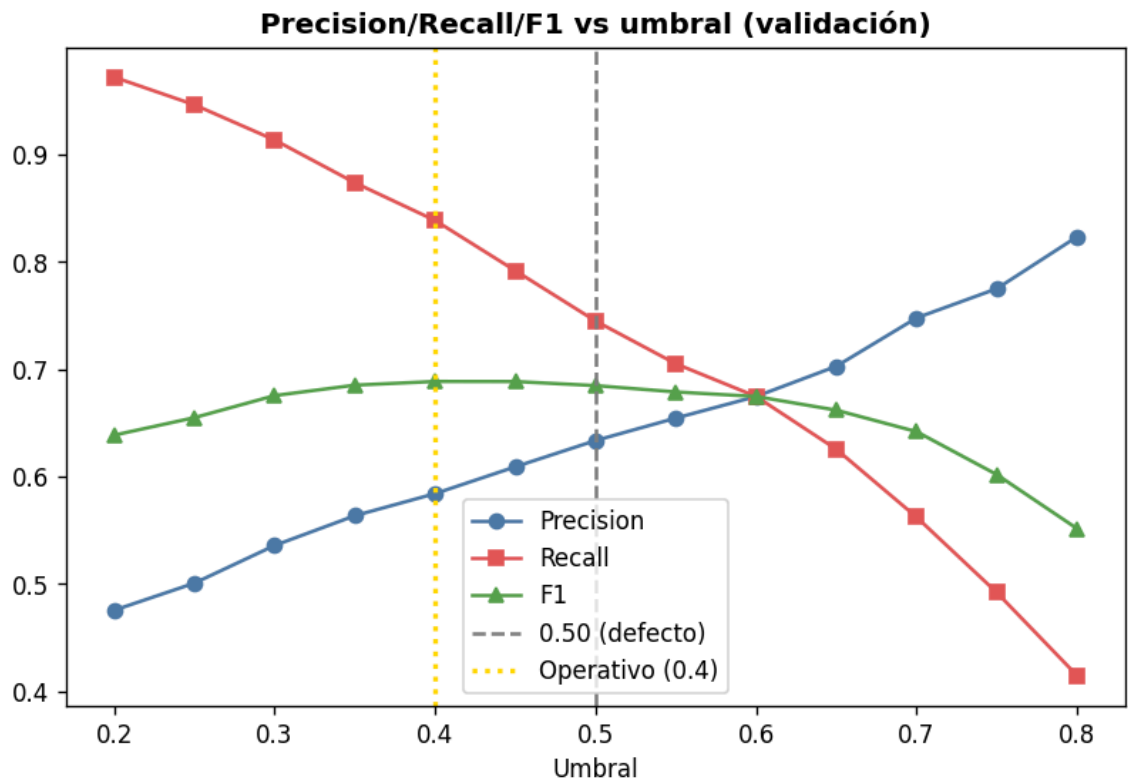
	position	cspm	earnedgpm	golddiffat15	xpdiffat15	y_true	y_pred	y_prob
1014	sup	8.193	453.402	852.8	-23.942	0	1	0.958
2255	top	9.547	433.667	691.782	-245.695	0	1	0.954
4974	top	5.689	316.189	-112.223	210.428	1	0	0.047
1326	top	7.143	361.87	-1452.33	408.477	1	0	0.052

Patrones: los **falsos negativos** son jugadores de LCK con oro/farmeo por debajo de lo típico de la liga (perfiles de soporte o partidas malas); los **falsos positivos** son jugadores de LCK CL con una gran partida (candidatos naturales a revisión de scouting). El error se concentra donde `earnedgpm` y `cspm` son ambiguos. Acciones: incorporar contexto de equipo y varias partidas por jugador (no una sola).

16. Optimización del umbral

Barrido en **validación** (no en test). Criterios: F1 óptimo = 0.475, Youden = 0.475, mínimo recall 0.90 → 0.31. **Umbral operativo elegido = 0.4** (*max F1 sujeto a recall* ≥ 0.80).

Umbral	Precision	Recall	F1-score	Falsos positivos	Falsos negativos
0.2	0.4754	0.9719	0.6385	458	12
0.25	0.5006	0.9461	0.6548	403	23
0.3	0.5357	0.9133	0.6753	338	37
0.35	0.5634	0.8735	0.685	289	54
0.4	0.584	0.8384	0.6885	255	69
0.45	0.609	0.7916	0.6884	217	89
0.5	0.6335	0.7447	0.6846	184	109
0.55	0.6543	0.7049	0.6787	159	126
0.6	0.6745	0.6745	0.6745	139	139
0.65	0.7026	0.6253	0.6617	113	160
0.7	0.7477	0.5621	0.6417	81	187
0.75	0.7749	0.4918	0.6017	61	217
0.8	0.8233	0.4145	0.5514	38	250



Qué prioriza: más **recall** que el 0.50 por defecto (no perder talento) sin desplomar la precisión; alineado con el 0.40 recomendado en el avance. **Costo que reduce:** falsos negativos (jugadores listos no detectados). **Consecuencia:** más falsos positivos (más jugadores enviados a revisión). **No debe adoptarse sin revisión organizacional:** el punto exacto depende de la capacidad de scouting y del costo real de cada tipo de error, que decide el área, no el modelo.

17. Modelo final

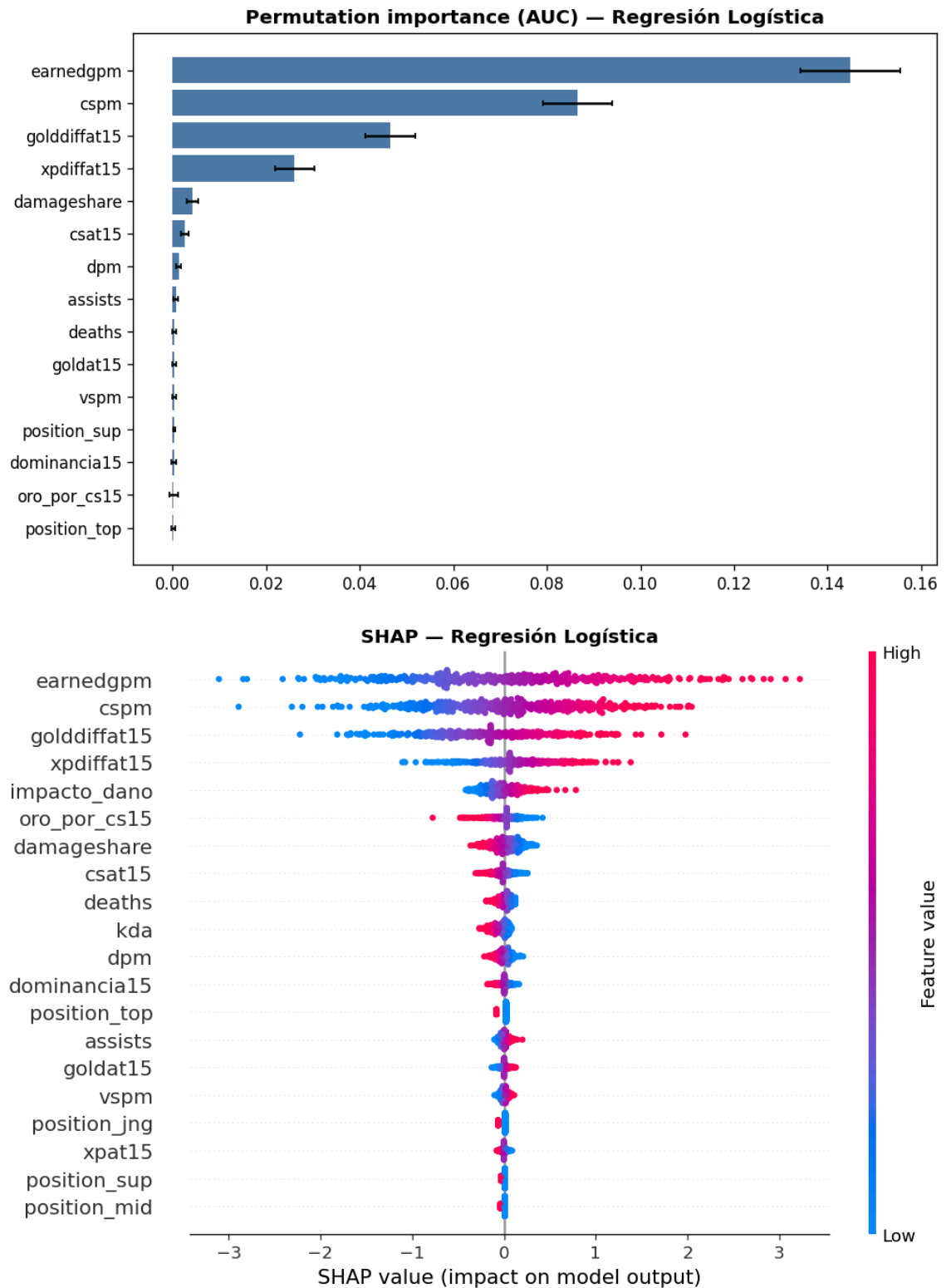
Criterio	Evidencia	Conclusión
Desempeño	F1 test 0.680, AUC 0.832, Recall 0.738	Supera con holgura al baseline
Generalización	Val F1≈0.692 → Test F1 0.680	Pérdida pequeña; sin sobreajuste
Errores	FP/FN interpretables (zona ambigua de oro/farneo)	Errores explicables y accionables
Interpretabilidad	Coeficientes directos + SHAP	Alta (defendible ante scouts)
Complejidad	Modelo lineal	Baja
Operación	Inferencia rápida, artefacto pequeño	Fácil de desplegar
Riesgo	Datos simulados; una partida por jugador	Uso solo como apoyo , con revisión humana

Modelo final: Regresión Logística. Es defendible para una **prueba controlada** no por tener la métrica más alta (el SVM lineal la iguala), sino por su **equilibrio**: desempeño competitivo, estabilidad, interpretabilidad, bajo costo y riesgo operativo, coherente con la regla de no elegir por métrica sola.

18. Interpretabilidad global

Coeficientes del modelo (features estandarizadas) + **permutation importance** (model-agnostic) + **SHAP** (`LinearExplainer`). Los tres coinciden: los factores dominantes son **earnedgpm** , **cspm** , **golddiffat15** y **xpdiffat15** — consistente con las hipótesis H1 (GPM/CSM mayores en LCK) y H2 (las diferencias a los 15 min sí importan).

Variable	Coeficiente	Coeficiente	:----- -----: -----:
earnedgpm	1.1156	1.1156	cspm 0.8662 0.8662 golddiffat15 0.6759 0.6759
xpdiffat15	0.4425	0.4425	impacto_dano 0.1904 0.1904 oro_por_cs15 -0.1517
	0.1517	damageshare -0.137 0.137 position_top -0.105 0.105 csat15 -0.0977	
	0.0977	position_jng -0.0766 0.0766 kda -0.0726 0.0726 deaths -0.0701 0.0701	
	dominancia15 -0.0657 0.0657 dpm -0.0649 0.0649 position_mid -0.0477 0.0477		
	goldat15 0.0472 0.0472 position_sup -0.0443 0.0443 assists 0.0439 0.0439		
	vspm 0.0375 0.0375 xpat15 -0.0279 0.0279 position_bot 0.0256 0.0256		
	deathsat15 -0.0148 0.0148 kills -0.0144 0.0144 assistsat15 -0.0099 0.0099		
	participacion15 -0.0073 0.0073 year -0.0064 0.0064 killsat15 0.0007 0.0007		



Limitaciones: la importancia/coeficiente indica asociación con la etiqueta, **no causalidad**; un coeficiente alto no significa que "subir esa métrica" convierta a un jugador en profesional de LCK.

19. Explicaciones individuales (VP, VN, FP, FN)

SHAP local para un caso de cada tipo. Sirve para **entender al modelo**, no para justificar automáticamente una decisión sobre el jugador.

Caso	Real	Predicción	Prob.	Factores principales (SHAP)
Verdadero positivo	LCK	LCK	0.88	cspm(+1.16); earnedgpm(+0.82); impacto_dano(-0.29); damageshare(+0.26)
Verdadero negativo	LCK CL	LCK CL	0.149	cspm(-1.34); xpdiffat15(+0.64); golddiffat15(-0.56); earnedgpm(-0.19)
Falso positivo	LCK CL	LCK	0.694	earnedgpm(+1.59); xpdiffat15(+0.50); oro_por_cs15(-0.45); cspm(-0.37)
Falso negativo	LCK	LCK CL	0.212	earnedgpm(-1.29); xpdiffat15(+1.19); cspm(-0.85); golddiffat15(-0.15)

20. Evaluación de sesgos (≥ 2 grupos)

Por posición y por nivel de gasto (earnedgpm):

Grupo	Tamaño	Tasa real (+)	Tasa pred (+)	Recall	Precision	FPR	FNR
bot	237	0.342	0.422	0.741	0.6	0.256	0.259
jng	237	0.392	0.464	0.817	0.691	0.236	0.183
mid	219	0.338	0.333	0.635	0.644	0.179	0.365
sup	257	0.358	0.471	0.815	0.62	0.279	0.185
top	250	0.348	0.392	0.69	0.612	0.233	0.31

Grupo	Tamaño	Tasa real (+)	Tasa pred (+)	Recall	Precision	FPR	FNR
bajo	400	0.138	0.082	0.327	0.545	0.043	0.673
medio	400	0.34	0.4	0.632	0.538	0.28	0.368
alto	400	0.59	0.772	0.907	0.693	0.579	0.093

Diferencias: el recall varía por posición (mid ≈ 0.64 vs. jng ≈ 0.82) y **fuertemente** por nivel de gasto (bajo ≈ 0.33 vs. alto ≈ 0.91). **Causa probable:** earnedgpm es el predictor dominante, así que los jugadores de LCK con bajo oro (p. ej. soportes) son más difíciles de detectar. **No se concluye discriminación** por una diferencia estadística: refleja la estructura de la señal y la calidad/representatividad de los datos (simulados). **Mitigación:** métricas por posición, umbral por rol, y más contexto (varias partidas, datos de equipo) antes de cualquier uso real.

21. Riesgos y condiciones de uso

Riesgo	Causa	Consecuencia	Prob.	Impacto	Mitigación
Falsos negativos	Zona ambigua de oro/farneo; soportes	Se pierde un jugador listo	Media	Alto	Umbral con recall alto; revisión humana
Falsos positivos	Buenas partidas puntuales en CL	Se evalúa a alguien no listo	Media	Medio	Revisión de scouting; varias partidas
Drift	Cambios de parche/meta temporada a temporada	Cae el desempeño	Alta	Alto	Monitoreo de distribución; reentrenar
Datos incompletos	Faltantes/errores de captura	Predicciones poco fiables	Media	Medio	Validación de entrada; imputación
Uso automático	Decidir solo con la predicción	Decisiones injustas/erróneas	Baja	Alto	Prohibido; siempre revisión humana
Sesgo	earnedgpm domina; roles distintos	Desempeño desigual por grupo	Media	Medio	Métricas por subgrupo; umbral por rol

Usos permitidos: apoyar la priorización de jugadores a revisar por el cuerpo de scouting.
Usos no recomendados: decidir ascensos/despidos, comparar salarios, o cualquier uso automático. **Revisión humana:** obligatoria en todos los casos. **Supervisión:** responsable de datos + coach de academia. **Suspender el modelo si:** cae el recall de forma sostenida, hay drift fuerte, o cambia la definición del problema.

22–23. Serialización y verificación de carga

Se serializa el **pipeline completo** (FeatureEngineer + preprocesamiento + modelo), reentrenado en **train+val**, con `joblib` , junto con metadatos (versión, umbral, features, hash SHA-256).

Artefacto	Archivo	Versión	Fecha	Biblioteca
Pipeline final	models/pipeline_final.joblib	1.0.0-fase-final	2026-09-29	joblib/sklearn 1.8.0
Metadatos	models/metadata.json	1.0.0-fase-final	2026-09-29	json
Umbral	models/metadata.json (campo 'umbral')	1.0.0-fase-final	2026-09-29	—

Seguridad: cargar `joblib` / `pickle` de fuentes desconocidas puede ejecutar código arbitrario. Cargar solo artefactos de origen confiable y

verificar el hash (se valida automáticamente al cargar).

El script **independiente** `src/predict.py` carga el artefacto, valida la entrada, aplica el umbral y devuelve la clasificación. Se verificó que las predicciones **coinciden** antes y después de serializar.

```
In [3]: from src.predict import cargar_modelo, predecir
art = cargar_modelo(verificar_hash=True)
print("Verificación de carga (predicciones idénticas):", R["verificacion_carga_ok"])
print("Modelo:", art["version"], "| umbral:", art["umbral"], "| fecha:", art["fecha"])
```

Verificación de carga (predicciones idénticas): True
Modelo: 1.0.0-fase-final | umbral: 0.4 | fecha: 2026-09-29

24–25. Mecanismo de inferencia y validación de entrada

`src/predict.py` acepta CSV/JSON y muestra: probabilidad, clasificación, umbral, versión/fecha y el mensaje *"Resultado estimado; requiere revisión humana."* La validación (`src/validate_input.py`) detecta campos faltantes, tipos incorrectos, valores fuera de rango, categorías desconocidas y columnas extra, **informando sin detener** la ejecución de forma inesperada.

Caso	Entrada	Resultado esperado	Resultado obtenido
Registro válido	completo y en rango	VÁLIDO, sin advertencias	VÁLIDO
Campo faltante	falta 'cspm'	ERROR: columna requerida	Falta la columna requerida: 'cspm'.
Categoría desconocida	position='jungla'	ADVERTENCIA (no bloquea)	Posición(es) no reconocida(s): ['jungla']. El OneHotEncoder las tratará como desconocidas (handle_unknown='ignore').
Tipo incorrecto	kills='cinco'	ERROR: no numérico	'kills': 1 valor(es) no numérico(s).
Valor extremo	cspm=99	ADVERTENCIA fuera de rango	'cspm': 1 valor(es) fuera del rango plausible [0, 14].

```
In [4]: from src.validate_input import validar_entrada
ejemplo = pd.read_csv("../examples/ejemplo_entrada.csv")
print(validar_entrada(ejemplo).resumen(), "\n")
predecir(ejemplo, art)[["probabilidad_LCK", "clasificacion", "umbral_utilizado", "mens
```

VÁLIDO

Out[4]:

	probabilidad_LCK	clasificacion	umbral_utilizado	mensaje
0	0.9942	LCK	0.4	Resultado estimado; requiere revisión humana.
1	0.0495	LCK CL	0.4	Resultado estimado; requiere revisión humana.

26. Simulación de inferencias (5 perfiles)

perfil	probabilidad_LCK	clasificacion	umbral_utilizado
Alto (perfil LCK)	0.9942	LCK	0.4
Bajo (perfil CL)	0.0117	LCK CL	0.4
Riesgo medio	0.2975	LCK CL	0.4
Categoría rara (pos. 'jungla')	0.1005	LCK CL	0.4
Cercano al umbral	0.4017	LCK	0.4

Uso en revisión humana: un perfil alto ($p \approx 0.99$) entra a la lista de candidatos prioritarios a revisar; uno cercano al umbral ($p \approx 0.40$) es un caso *dudoso* que **siempre** requiere ojo humano; uno bajo ($p \approx 0.01$) tiene baja prioridad, pero **no** se descarta automáticamente. La categoría rara (posición desconocida) se procesa sin romper. **No se automatiza** ninguna decisión de ascenso con estos números.

27. Plan de monitoreo

Indicador	Frecuencia	Umbral de alerta	Acción
Valores faltantes	Por lote	> 15 % en una variable	Revisar captura; pausar si es crítico
Recall (clase LCK)	Mensual / por split	Caída > 5 pts vs. validación	Recalibrar umbral / reentrenar
Data drift (PSI)	Mensual	PSI > 0.2 en variables clave	Investigar meta/parche; reentrenar
Tiempo de respuesta	Continuo	> 500 ms por lote	Revisar infraestructura
Categorías nuevas	Por lote	Cualquier <code>position</code> desconocida	Revisar diccionario; reentrenar si persiste

Cubre calidad de datos, distribución, desempeño (precision/recall/F1/AUC/FPR/FNR) y operación (tiempo, errores, disponibilidad, versión activa). Puede implementarse como reporte periódico; no requiere plataforma productiva.

28. Criterios de actualización

Situación	Criterio concreto	Quién revisa	Acción
Caída de recall	> 5 pts sostenida (2 periodos)	Responsable de datos	Recalibrar umbral / reentrenar
Aumento de FN	FN por encima del objetivo del área	Coach + datos	Revisar umbral; reentrenar
Drift de distribución	PSI > 0.2	Responsable de datos	Reentrenar con datos recientes
Categorías nuevas	Nueva liga/rol no visto	Responsable de datos	Actualizar pipeline y reentrenar
Cambio de negocio	Cambia la definición de "listo"	Dirección deportiva	Rediseñar el problema
Diferencias por grupo	Brecha excesiva de recall entre roles	Comité de revisión	Mitigar / suspender
Pérdida de disponibilidad	Servicio caído	Operación	Suspender y usar proceso manual

29. Trazabilidad final

Elemento	Detalle
Versión del dataset	simulado_v1 (8000×20), semilla 42
Fuente	Simulación reproducible (<code>src/data_simulada.py</code>); inspirada en Oracle's Elixir (LCK/LCK CL 2022–2024)
Fecha de consulta	2026-09-29
Variables utilizadas	17 numéricas base + <code>position</code> + 5 derivadas
Variables excluidas	<code>league</code> , <code>gameid</code> , <code>playerid</code> , <code>teamid/team</code> (fuga)
Pipeline	FeatureEngineer → ColumnTransformer (num: mediana+Z; cat: moda+OHE)
Modelos evaluados	Dummy, LogReg, Árbol, RF, GB, HistGB, SVM (lin/RBF), MLP
Hiperparámetros	RF <code>{'n_estimators': 200, 'min_samples_leaf': 8, 'max_features': 'sqrt', 'max_depth': 6}</code> ; SVM <code>{'C': 0.1, 'kernel': 'linear'}</code>
Métricas de validación	F1≈0.692, AUC≈0.844 (LogReg, CV)
Métricas de prueba	F1=0.680, AUC=0.832, Recall=0.738

Elemento	Detalle
Umbral	0.4 (max F1 sujeto a recall>=0.80)
Modelo final	Regresión Logística
Archivo serializado	<code>models/pipeline_final.joblib</code> (+ <code>metadata.json</code> con hash)
Bibliotecas	scikit-learn 1.8.0, Python 3.11.15 (ver <code>requirements.txt</code>)
Fecha de entrenamiento	2026-09-29
Archivo principal	<code>notebooks/01_fase_final.ipynb</code> / <code>src/run_pipeline.py</code>
Responsable	Felix Yael

30. Conclusiones personales

Sobre el modelo. Un modelo lineal simple (regresión logística) igualó en F1 y AUC a alternativas mucho más complejas (Random Forest, Gradient Boosting, HistGradientBoosting, SVM-RBF y una red neuronal). Para mí la lección es clara: en este problema la señal es fundamentalmente **lineal y separable** con las variables de economía temprana, de modo que la complejidad extra no aportaba desempeño y sí restaba transparencia. En un contexto de scouting —donde hay que **defender** cada decisión ante un coach— preferí la interpretabilidad antes que ganar un decimal de métrica con una caja negra.

Las variables dominantes (`earnedgpm` , `cspm` , `golddiffat15` , `xpdiffat15`) tienen sentido de juego. La diferencia entre un jugador de LCK y uno de Challengers no está tanto en las *kills* como en la **eficiencia económica y el control de la fase temprana**: farmear más por minuto, convertir ese farmeo en oro y llegar al minuto 15 con ventaja de oro y de experiencia. El modelo, sin saber nada de League of Legends, redescubrió un principio que cualquier analista reconocería: en el nivel alto se gana el juego temprano de recursos.

Sobre las decisiones de diseño. Prioricé el **recall** (umbral operativo 0.40 en lugar de 0.50) porque en scouting el error caro es el **falso negativo**: descartar a un jugador que sí estaba listo. Revisar de más a alguien (falso positivo) solo cuesta tiempo de análisis; perder un talento cuesta mucho más. Esa asimetría de costos —y no una métrica abstracta— fue la que fijó el umbral.

El análisis de sesgos me dejó la conclusión más incómoda y más valiosa: el recall cae a ~0.33 en jugadores de bajo `earnedgpm` y sube a ~0.91 en los de alto gasto, y los soportes quedan peor clasificados. El modelo no discrimina por malicia; simplemente **penaliza a los roles de bajo oro** porque su mejor predictor es económico. Esto me convence de que una métrica única no captura el valor real de un soporte o de un jungla de bajo recurso, y de que cualquier uso serio exigiría **normalizar o fijar umbrales por rol** antes de comparar jugadores.

Sobre los límites y el uso responsable. Trabajé con datos **simulados** y con **una sola partida por jugador**. El juego real es otra cosa: consistencia a lo largo de una temporada, sinergia de equipo, adaptación al parche y al meta, comunicación y mentalidad —nada de eso está en las estadísticas. Por eso insisto en que este modelo es un **apoyo para priorizar a quién observar**, nunca un veredicto sobre un jugador.

Lo que más valoro del proyecto no es la métrica, sino la **trazabilidad**: semilla fija, exclusión explícita de `league` para evitar fuga de información, reproducción bit a bit contra el avance, hash SHA-256 del artefacto y verificación de que las predicciones coinciden después de serializar. Aprendí que un modelo defendible se sostiene tanto en el **proceso** como en el resultado.

Trabajo futuro. El siguiente paso natural sería llevar esto a **datos reales de la Riot API** (Challengers del ladder en cualquier servidor), con la advertencia de que el soloqueue no equivale al juego profesional: habría que reetiquetar y medir "*parecido al rendimiento pro*" en lugar de prometer que alguien "será pro". Es un problema más difícil, pero también más honesto, y es la dirección en la que me gustaría continuar.

— **Felix Yael**

Reproducción completa

```
pip install -r requirements.txt
python -m src.run_pipeline          # regenera tablas, figuras y el
artefacto
python -m src.predict --input examples/ejemplo_entrada.csv
Documentación complementaria: reports/ficha_modelo.md (técnica) y
reports/reporte_ejecutivo.md (no técnica). Recordatorio: el resultado del modelo es
un apoyo; toda decisión requiere revisión humana.
```

In []:

In []: